

PYTHON PROGRAMMING PROJECT

STUDENT RECORDS SYSTEM

A Simple Python Menu-Driven Program to Manage Student Marks and Courses

Submitted By:	Akarsh Yadav
Registration No:	26BCE10010
Course:	B.Tech CSE - Core
Subject:	Programming in Python
Platform:	VITyarthi Portal
Date:	Academic Year 2026

2. INTRODUCTION

In our college, keeping track of student details, enrolled subjects, and exam grades manually on paper can be confusing and time-consuming. To solve this, I built a simple **Student Records System** using basic Python programming.

This project uses basic Python features like functions, lists, dictionaries, and a while loop to create an easy menu. It allows the user to add new students, enter their grades, check which courses they are taking, and view all student details in one place.

3. PROBLEM STATEMENT

In school or college, managing student data manually creates several basic problems:

- It takes a lot of time to write down every student's subject list manually.
- Sometimes grades are accidentally assigned to subjects that the student is not taking.
- Searching for all students enrolled in a specific course requires going through every record one by one.

4. FUNCTIONAL REQUIREMENTS

No.	Feature Name	What It Does in the Program
1	Add Student	Takes student name, register number, and course list from user and saves them.
2	Add Grade	Checks if the student takes that subject and adds the grade (like A, B+).
3	Add Extra Course	Adds a new course name to an already registered student's course list.
4	Check Enrollment	Tells whether a specific student is taking a particular course or not.
5	List Students by Course	Shows the names of all students who are taking a specific subject.

5. NON-FUNCTIONAL REQUIREMENTS

- **Easy to Use:** Simple numbered menu (1 to 7) so anyone can type a number and run options easily.
- **Quick Response:** Performs all actions instantly since data is stored directly in Python memory.
- **Basic Checks:** Prevents entering grades for subjects the student has not enrolled in.
- **Runs Anywhere:** Can run on any computer having standard Python 3 installed.

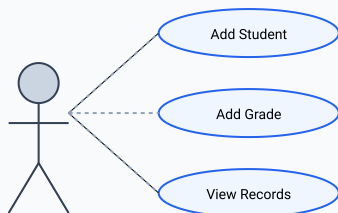
6. SYSTEM ARCHITECTURE

The program works in a straightforward, step-by-step way. When the user types a menu choice in the terminal, the Python script calls the specific function, processes the input, and updates the dictionary stored in memory.

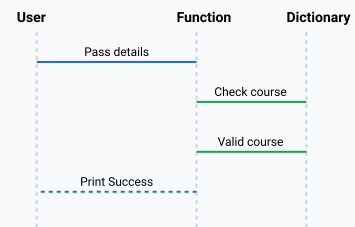


7. DESIGN DIAGRAMS

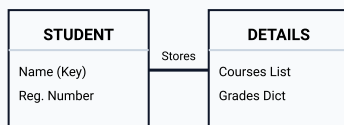
A. PROGRAM USER OPTIONS DIAGRAM



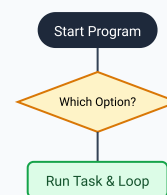
B. ADDING GRADE STEP-BY-STEP



C. HOW DATA IS CONNECTED



D. MAIN PROGRAM LOOP



8. DESIGN DECISIONS & MY APPROACH

- **Why I used a Python Dictionary:** Using a dictionary with student names as keys made it easy to group a student's register number, course list, and grades all together in one variable.
- **Why I used Functions:** Writing separate functions like `add_student()` and `add_grade()` kept the main menu clean and easy to read.
- **Using `.strip()`:** I added `.strip()` to user inputs so extra spaces typed by mistake don't mess up student or course names.

9. IMPLEMENTATION DETAILS

The code is written in basic Python. Here are two main functions used in the project:

```
def add_student(name, Reg, courses): if name in student_records: print(f"Student '{name}'  
already exists.") return # Saving student details in dictionary student_records[name] = {"Reg.No":  
Reg, "grades": {}, "courses": courses} print(f"Student '{name}' added successfully.") def  
add_grade(name, course, grade): if name not in student_records: print(f"Student '{name}' not  
found.") return # Checking if student is enrolled before adding grade if course not in  
student_records[name]["courses"]: print(f"Student '{name}' is not enrolled in {course}.") return  
student_records[name]["grades"][course] = grade print(f"Grade {grade} added for {course} for  
student '{name}'.")
```

10. OUTPUT SCREEN RESULTS

```
Welcome to the Student Records System!  
Menu:  
1. Add a new student | 2. Add a grade | 3. View all student records  
  
Enter your choice: 1  
Enter student name: Rahul Sharma  
Enter student Reg.No: 26BCE10112  
Enter courses separated by commas: Mathematics, Physics  
Student 'Rahul Sharma' added successfully.  
  
Enter your choice: 2  
Enter student name: Rahul Sharma  
Enter course name: Mathematics  
Enter grade: A+  
Grade A+ added for Mathematics for student 'Rahul Sharma'.
```

11. TESTING APPROACH

I tested the code by trying different inputs in the screen menu to check if it handles mistakes properly:

Test No.	What I Tried	What Should Happen	Result
1	Added same student name twice	Showed "Student already exists" message	WORKING
2	Added grade for wrong course	Showed "Student is not enrolled in course"	WORKING
3	Searched course with no students	Showed "No students found for this course"	WORKING

12. CHALLENGES I FACED

- **Commas in Course Names:** When taking multiple courses at once, splitting inputs by comma left extra spaces (e.g., " Physics"). I fixed this using `course.strip()`.
- **Data Resetting on Exit:** Since all data is stored in standard memory, everything resets when the program stops running.

13. LEARNINGS & KEY TAKEAWAYS

Through this project, I learned several fundamental Python programming concepts:

- **Working with Dictionaries:** I learned how to store nested data inside a Python dictionary using keys and values.
- **Creating Functions:** I understood how dividing code into smaller functions makes it much easier to write and debug.
- **Using Loops for Menus:** I learned how to use a `while True` loop so the user can keep choosing options until they select 'Exit'.

14. FUTURE ENHANCEMENTS

Since I am currently in my 1st year, I plan to improve this project further as I learn more topics:

1. **Save Data to a File:** Add file handling (or JSON) so that student details remain saved even after closing the program.
2. **Use Registration Number as Primary Key:** Use register numbers instead of names to avoid conflicts if two students have the same name.
3. **Ignore Capitalization Errors:** Use `.lower()` so typing "math" or "Math" matches correctly.

15. REFERENCES

- VITYarthi Learning Platform – Python Course Notes for 1st Year Students.
- Classroom lectures and lab exercises on Functions and Loops.

Evaluator Remarks:

Signature: _____

Final Grade / Marks:

Date: ____ / ____ / 2026